

Opis problemu

Celem jest napisanie programu który będzie produkował plik wynikowy csv gdzie będą zapisane wyniki symulacji MonteCarlo po przez skanowanie przestrzeni parametrów.

Pipeline

1. Czytanie pliku konfiguracyjnego `.json`
2. Generacja przestrzeni parametrów na podstawie specyfikacji
3. Generacja krzywych IV na podstawie listy parametrów i wybranego modelu
4. Policzenie błędu χ^2
5. Porównanie błędów χ^2 ze statystyką testową i odrzucenie punktów w których błąd jest większy niż odpowiednia wartość
6. Generacja i zapis obrazków z elipsoidami kowariancji dla 1,2 i 3 sigma.
7. Zapis wygenerowanych danych do pliku csv wraz z błędem oraz z policzoną ilością stopni swobody

Numeryka i kod w C++

1. Na razie aplikacja powinna wspierać dwa modele prądu. 4 oraz 6 parometrowy, skrypt. Dobrze by było aby można łatwo było dodać potem nowe modele na podstawie tylko implementacji matematycznej funkcji. Aby policzyć prąd w 4 parometrowym modelu należy rozwiązać poniższe równanie: $I(V) = I_0 e^{\frac{V - I R_s}{A k T}} - \frac{V - I R_s}{R_{sh}}$ Dla modelu sześćo parametrów równanie wygląda następująco: $I(V) = I_0 e^{\frac{V - I R_s}{A k T}} - \frac{V - I R_s}{R_{sh}} - \frac{(V - I R_s)^\alpha}{R_{sh2}}$ Aby rozwiązać to równanie wykorzystuje się funkcję Lamberta W. Poniżej załączam kod który na wejściu przyjmuje parametry a zwraca prąd:

```
// 4p
const double k = 8.6e-5; // Już jest w eV

double FourPModel(double V, double I0, double A, double Rsh, double Rs, double T)
{
    double x = ((I0 * Rs) / (A * k * T)) * std::exp(V / (A * k * T));
    double I_lw = utl::LambertW<0>(x);
    I_lw *= (A * k * T) / Rs;
    return I_lw + (V - I_lw * Rs) / Rsh;
};

// /6p
double SixPModel(double V, double I0, double A, double Rsh, double Rs, double alpha, double Rsh2, double T)
{
    double x = ((I0 * Rs) / (A * k * T)) * std::exp(V / (A * k * T));
    double I_lw = utl::LambertW<0>(x);
    I_lw *= (A * k * T) / Rs;
    double additionalFactor = std::pow((V - I_lw * Rs), alpha) / Rsh2;
    return I_lw + (V - I_lw * Rs) / Rsh + additionalFactor;
};
```

Implementacja funkcji `utl::LambertW<0>` znajduje się [tutaj](#)

2. Aby policzyć błąd „dopasowania” krzywej należy skorzystać ze statystyki χ^2 . **Noise** jest parametrem procentowym szumu zapisanym w konfiguracji symulacji. Dla 1% powinien on przyjąć wartość `0.01`.

$$\Delta\chi^2 = \sum_n \frac{(I_{\text{true}} - I_{\text{tested}})^2}{\text{Noise}}$$

Poziom ufności	p=1	p=2	p=3	p=4	p=5	p=6
68.3%	1.00	2.30	3.53	4.72	5.89	7.04
90%	2.71	4.61	6.25	7.78	9.24	10.64
95.4%	4.00	6.18	8.02	9.72	11.31	12.85
99%	6.63	9.21	11.34	13.28	15.09	16.81
99.73%	9.00	11.83	14.16	16.25	18.21	20.06
99.9%	10.83	13.82	16.27	18.47	20.52	22.46

1. Aby policzyć ilość stopni swobody jest to ilość parametrów modelu - ilość zafiksowanych parametrów - 1

Konfiguracja

Konfiguracja będzie zapisana w pliku `json`

```
{
  "model": 4,
  "path_to_characteristic": "D:/some path",
  "noise": 1e-2, //-? 1%
  "number_of_steps": 1e5,
  "true_parameters": {
    "A": 1.533544e0,
    "I0": 5.011808e-9,
    "Rs": 7.613373e+00,
    "Rsh": 7.705294e+03,
  },
  "bounds":{
    "A": [0.5, 2.5],
    "I0": [1e-10,9e-7],
    "Rs": [1e-1,9e2],
    "Rsh": [1e2,9e4],
  },
  "fixed_parameters":{
    // "A": 1.533544e0,
    // "I0": 5.011808e-9,
```

```

    // "Rs": 7.613373e+00,
    // "Rsh": 7.705294e+03,
  },

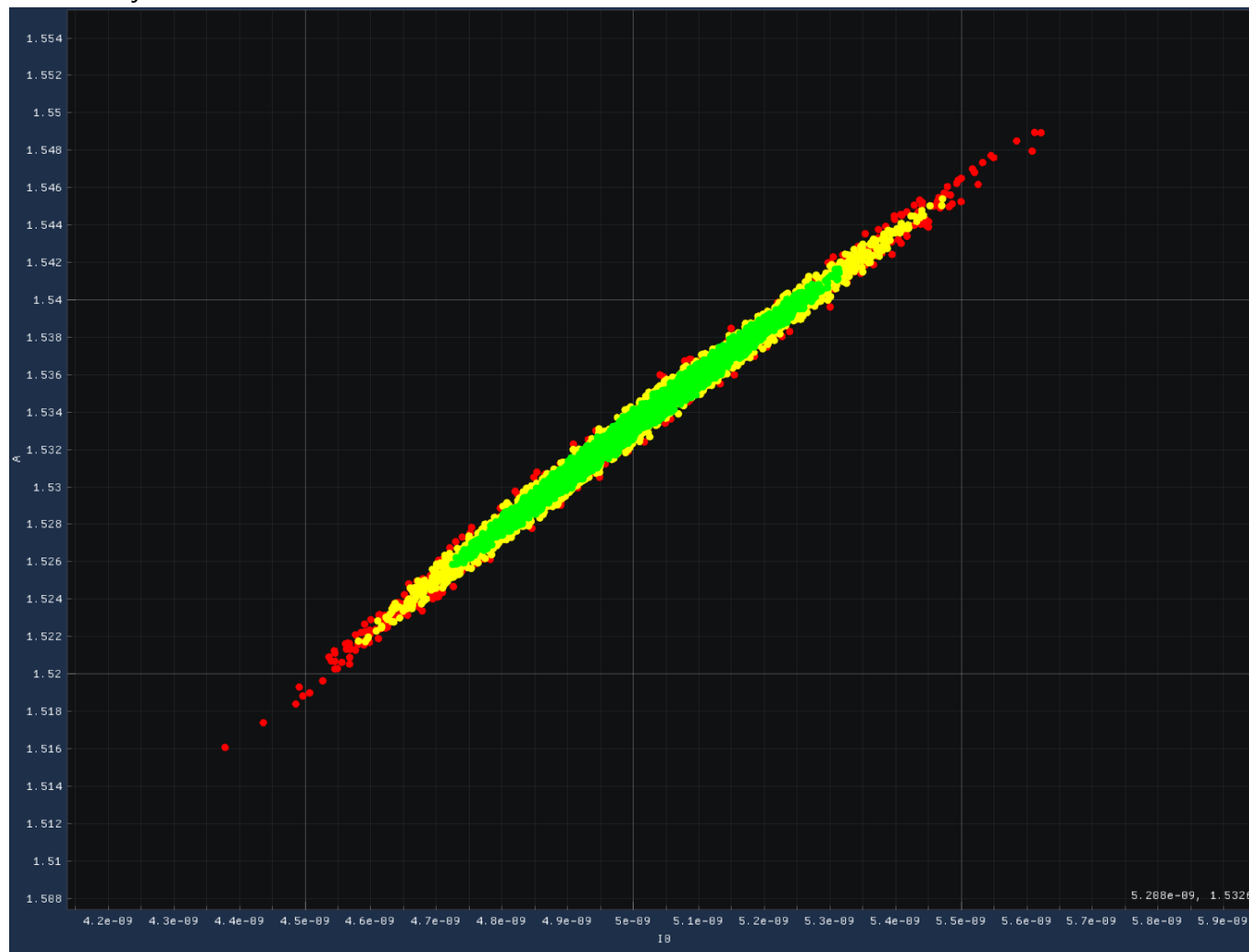
}

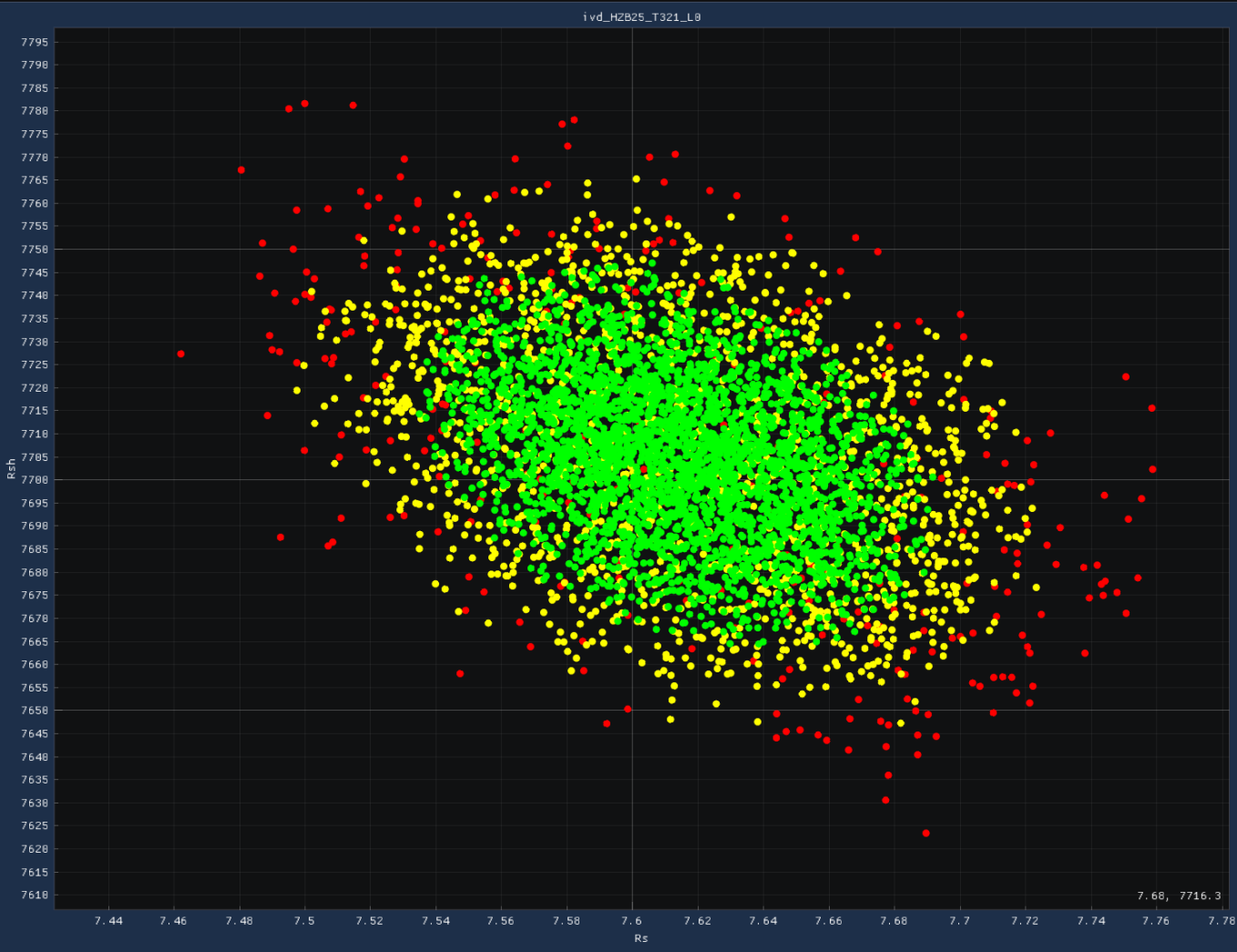
```

granice podane są generowane automatycznie, mogą być podawane przez użytkownika ale z punktu widzenia działania programu nie ma to większego znaczenia. W przypadku gdzie granice różnią się o kilka rzędów wielkości to należy wypraktykować jak będzie najlepiej generować dane, z jakiego rozkładu. Może być tak, że parametr: `number_of_steps` będzie na każdą dekadę albo trzeba by podzielić `number_of_steps` równo na każdą dziesiątkę dekady aby całościowo uzyskać rozkład jednorodny

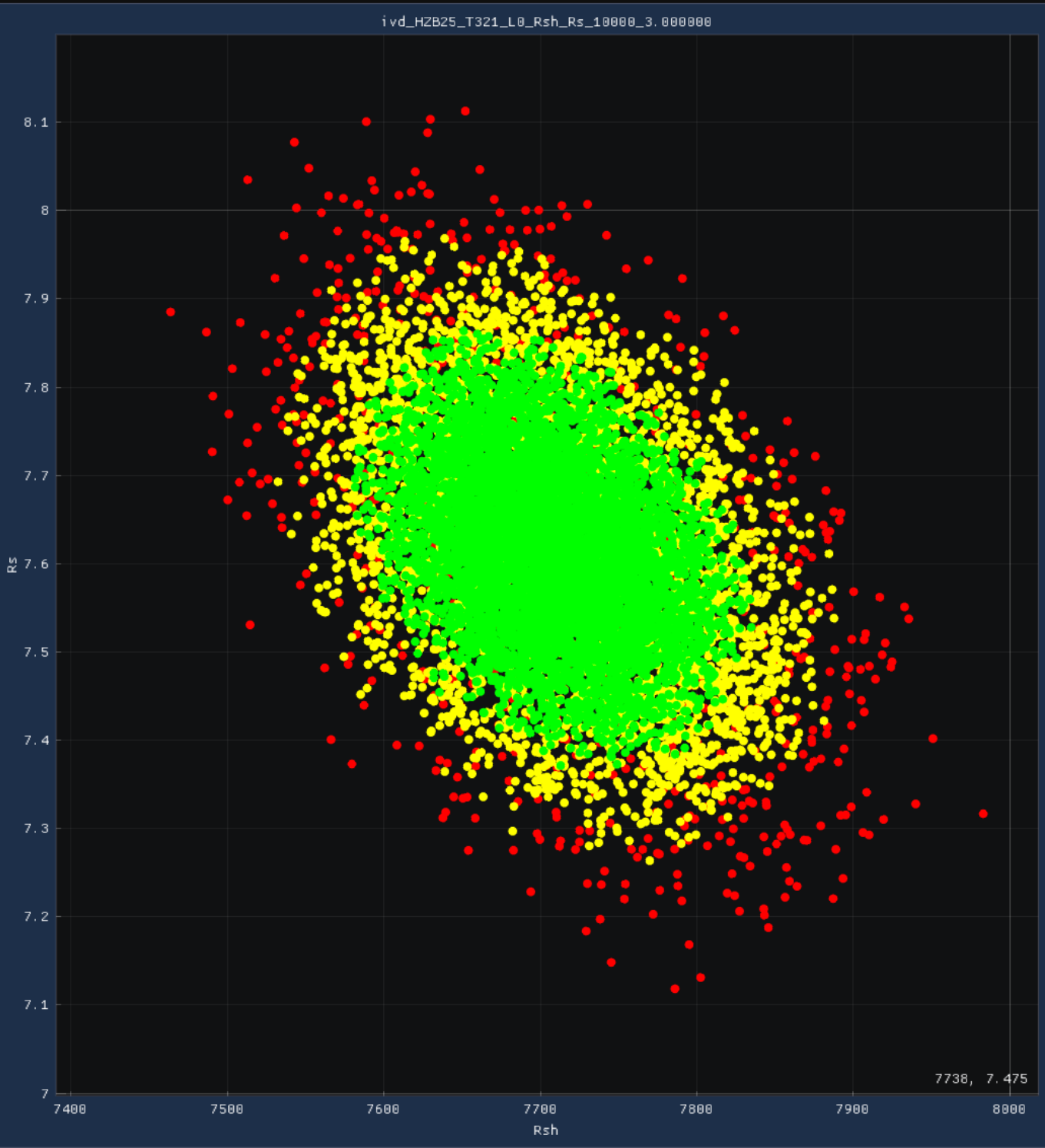
Wyniki

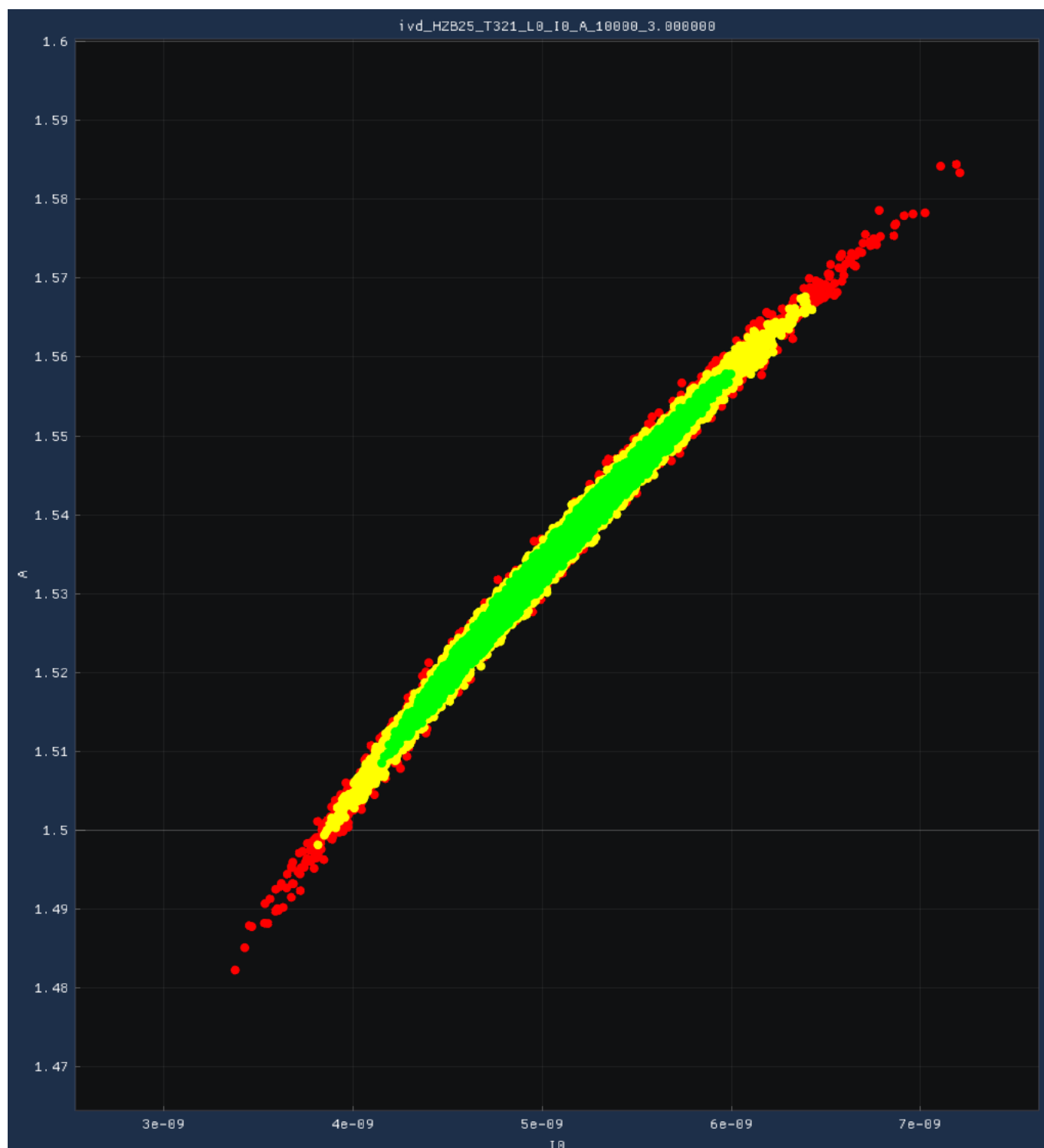
5000 iteracji 1% szumu





10_000 iteracji 3% szumu





należy pamiętać, że wyniki otrzymane przez opisany wyżej algorytm będą się trochę różnić od tych z aplikacji, ze względu na inne rozłożenie próbkowania przestrzeni parametrów, natomiast jakościowo rezultat powinien być ten sam.